PyOpenGL: Introduction

How a Computer Draws a Simple Square on the Screen

Introductory Session

August 12, 2026

What We Will Learn Today

- What OpenGL and PyOpenGL are, in everyday words
- How a computer screen actually shows a shape
- What each line of our small program is doing
- How to change the program to draw your own shapes

Part 1: The Basic Idea

How does a computer shows a shape ?

. The computer's graphics processor figures out which pixels map to the edges and interior of the shape, lighting up specific red, green, and blue sub-pixels to form the visual image.

Filling the Shape

- **Rasterization:** The computer decides which pixels sit inside the shape lines.
- **Coloring:** It fills those inner pixels with the correct color.

What is OpenGL?

OpenGL is a set of ready-made instructions that a program uses to tell the computer: *"please draw this on the screen."*

Think of it like ordering food at a restaurant:

- You (the program) do not cook.
- You just instruct what you want.
- The kitchen (your graphics card) prepares it.

OpenGL is the **menu and the language** you use to place the order.

What OpenGL Can and Cannot Do

It CAN

- Draw points, lines, shapes
- Fill them with colour
- Rotate and move them

It CANNOT

- Open a window on your desktop
- Notice mouse clicks or key presses
- Play sound or open image files

So OpenGL is only the **painter**.
We still need someone to give the painter a **canvas and a frame** — that helper is called **GLUT**.

What is PyOpenGL?

PyOpenGL platform independent. Just like WORA.

PyOpenGL is simply OpenGL made usable from **Python** — a much easier, friendlier language.

Simple way to remember

OpenGL = the drawing tool
Python = the easy language
PyOpenGL = using the drawing tool from Python

We install it by typing one line in the terminal:

```
pip install PyOpenGL PyOpenGL_accelerate
```

The Three Lines at the Top

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
```

These lines bring in three toolboxes:

Toolbox	Its job (in simple words)
GL	The **paintbrush** — draws shapes and colours
GLUT	The **window** — makes the box on your screen
GLU	**Extra helpers** — shortcuts for harder shapes

Setting the Window Size

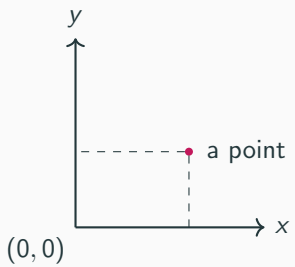
```
w, h = 500, 500
```

- `w` means **width**, `h` means **height**.
- Both are 500 **pixels** — so our window is a square box, 500 tiny dots wide and 500 tall.
- A **pixel** is one tiny dot of light on your screen. Your whole screen is made of millions of them.

We write the size once at the top so that later, if we want a bigger window, we only change this one line.

Part 2: Drawing the Square

How the Computer Locates the projection Window size



w and h represent the window size.

- w = width of the window (horizontal direction)
- h = height of the window (vertical direction)
- $(0, 0)$ is the **bottom-left** corner

Careful: in OpenGL, y grows **upward**.

Section 1: The `square()` Function

```
def square():
    glBegin(GL_QUADS) # GL_QUADS is a drawing mode (primitive type). It
        instruct OpenGL to Group every 4 vertices together
    glVertex2f(100, 100) # corner 1: bottom left
    glVertex2f(200, 100) # corner 2: bottom right
    glVertex2f(200, 200) # corner 3: top right
    glVertex2f(100, 200) # corner 4: top left
    glEnd() # finished
```

Reading it out loud:

> *"I am about to draw a four-sided shape. Here is corner one, corner two, corner three, corner four. Done — now draw it."*

GL_QUADS and GL_TRIANGLES etc Used defined or system defined ?

These are **These are System defined ready-made names given by OpenGL** — we do not create them. They arrive with the very first line of our program:

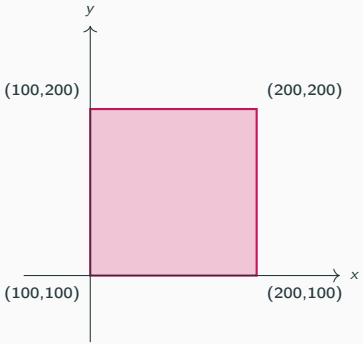
```
from OpenGL.GL import * # brings in all OpenGL names
```

Each name secretly stands for a number, e.g. GL_QUADS is 0x0007. **You never need to remember the number** — just use the name.

Name	What it draws	Corners needed
GL_POINTS	separate dots	1 each
GL_LINES	separate lines	2 each
GL_LINE_STRIP	lines joined end to end	many
GL_TRIANGLES	triangles	3 each
GL_QUADS	four-sided shapes	4 each
GL_POLYGON	one many-sided shape	3 or more

So glBegin(GL_QUADS) simply means: **"the corners I am about to give you belong to a four-sided shape."**

The Square We Just Made



- Bottom-left corner sits 100 dots right and 100 dots up.
- Each side is $200 - 100 = 100$ dots long.
- So it is a small square, slightly away from the corner of the window.

Try it

Change all the 200s to 300 and the square becomes bigger.

Why Corner Order Matters

We listed the corners going **around the edge**:
bottom-left $\rightarrow$ bottom-right $\rightarrow$ top-right $\rightarrow$ top-left.

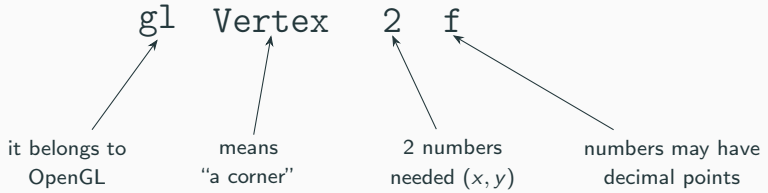
This is like drawing with a pencil without lifting it.

If you jump about randomly — say bottom-left, top-right, bottom-right, top-left — you get a crossed "bow-tie" shape instead of a square.

Rule of thumb

Always go around the shape in one direction, like walking around a room along the walls.

Understanding the Strange Names



So `glVertex2f(100, 100)` simply means: **"here is one corner, at position 100 across and 100 up."**

Part 3: Setting Up the View

Section 2: The `iterate()` Function

```
def iterate():
    glViewport(0, 0, h, w)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glOrtho(0.0, 500, 0.0, 500, 0.0, 1.0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
```

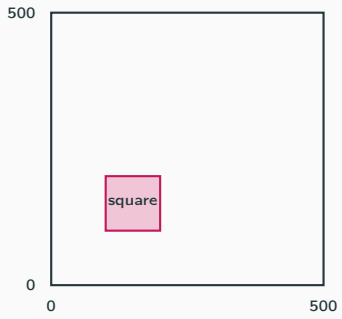
This looks scary, but it answers only one simple question:

> *"Which part of the window do we draw on, and how big is one unit?"*

It is like setting up a camera on a tripod before taking a photo.

Understanding `glOrtho`

```
glOrtho( left, right, bottom, top, near, far )
glOrtho( 0.0, 500, 0.0, 500, 0.0, 1.0 )
```



The whole window, with the edges named

It simply tells OpenGL **what numbers to write on the ruler**.

- left edge is called **0**, right edge **500**
- bottom edge is **0**, top edge **500**
- the last two numbers are for 3D depth — ignore them here

Our window is 500 *pixels* and our area is 500 *units*, so:

1 unit = 1 dot on screen

Try it: smaller numbers (250) make everything look **twice as big**; larger numbers (1000) make everything look

The Same Lines, Explained One by One

Line	What it means
`glViewport(...)`	"Use this rectangle of the window as my drawing paper." Here it is the whole window.
`glMatrixMode(GL_PROJECTION)`	"I now want to adjust the **camera lens**."
`glLoadIdentity()`	"Erase my earlier settings and start fresh." Like clearing a whiteboard.
`glOrtho(0, 500, 0, 500, ...)`	"My drawing area goes from 0 to 500 across and 0 to 500 up."
`glMatrixMode(GL_MODELVIEW)`	"Now I want to adjust **where the objects sit**."
`glLoadIdentity()`	"Start fresh here too."

Part 4: Showing It on Screen

The `showScreen()` Function

```
def showScreen():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    iterate()
    glColor3f(1.0, 0.0, 3.0)
    square()
    glutSwapBuffers()
```

This is the **main painting routine**. It runs again and again, and each time it repaints the whole picture from scratch.

Reading It Like a Recipe

1. `glClear(...)` — **Wipe the canvas clean.** Remove whatever was drawn before.
2. `glLoadIdentity()` — **Reset positions**, so nothing from last time carries over.
3. `iterate()` — **Set up the camera** again (the function we just learned).
4. `glColor3f(...)` — **Pick up a coloured brush.**
5. `square()` — **Draw the square** with that brush.
6. `glutSwapBuffers()` — **Show the finished picture** to the viewer.

Important habit of OpenGL

You must choose the colour **before** you draw the shape — exactly like dipping your brush in paint before touching the paper.

Choosing a Colour

```
glColor3f(1.0, 0.0, 3.0)
```

The three numbers are **Red**, **Green**, **Blue**. Each should be between **0** (none) and **1** (full).

Numbers	**Colour**	
`1.0, 0.0, 0.0`	pure red	
`0.0, 1.0, 0.0`	pure green	
`1.0, 1.0, 0.0`	yellow	(red + green)
`1.0, 0.6, 0.8`	pink	

Part 5: Making the Window Appear

The Last Block of Code

```
glutInit()
glutInitDisplayMode(GLUT_RGBA)
glutInitWindowSize(w, h)
glutInitWindowPosition(0, 0)
wind = glutCreateWindow(b"OpenGL Coding Practice - Simple Square")
glutDisplayFunc(showScreen)
glutIdleFunc(showScreen)
glutMainLoop()
```

Everything above was **preparation**.

These lines actually **open the window and start the program**.

Line by Line, in Plain Words

Line	Meaning
`glutInit()`	"Wake up and get ready."
`glutInitDisplayMode(...)`	"I want a full-colour window."
`glutInitWindowSize(w, h)`	"Make it 500 by 500."
`glutInitWindowPosition(0,0)`	"Put it at the top-left of my screen."
`glutCreateWindow(...)`	"Open the window, with this title on top."
`glutDisplayFunc(showScreen)`	"Whenever you need to paint, call my `showScreen` function."
`glutIdleFunc(showScreen)`	"When you have nothing else to do, paint again."
`glutMainLoop()`	"Now keep running until I close the window."

Two Small Curiosities

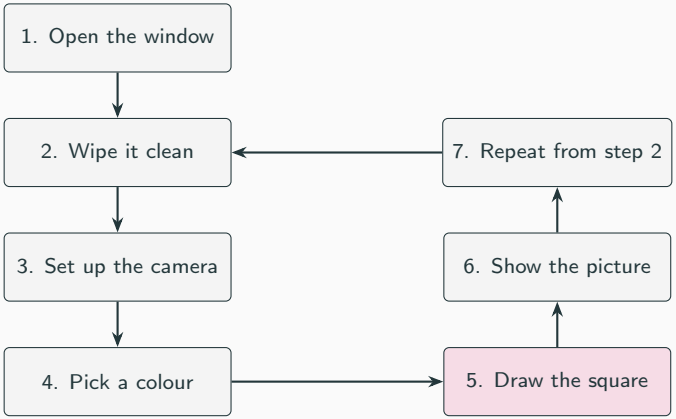
Why the letter `b` before the title?

`b"Simple Square"` — the `b` tells Python to hand the text over in the older style that OpenGL expects. Without it, the program may complain. Just remember to keep it there.

Why does `glutMainLoop()` come last?

Because it never finishes. It keeps the window alive and waits for you. Anything written after this line will simply never run.

The Whole Story in One Picture



Part 6: Try It Yourself

Easy Things to Change

Change this	And you will see
`glColor3f(0.0, 0.0, 1.0)`	A blue square
The corner numbers	A bigger, smaller, or shifted square
`GL_QUADS` → `GL_LINE_LOOP`	Only the outline, not filled in
`GL_QUADS` → `GL_POINTS`	Just four tiny dots at the corners
Remove one `glVertex2f` line	A triangle instead of a square
`w, h = 800, 600`	A wider window

The best way to learn graphics is to **change one thing at a time** and see what happens.

Words You Now Know

Word	Simple meaning
Pixel	One tiny dot on the screen
Vertex	A corner of a shape
Window	The box your drawing appears in
Viewport	The part of the window you draw on
Buffer	A page held in memory
Swap buffers	Show the finished page
Callback	A function the system calls for you
Loop	Doing something over and over

Problem Statement

The scene should contain the following objects:

1. **Sky:** Create a blue background area using a rectangle.
2. **Ground:** Create a green ground region using a rectangle.
3. **Sun:** Create a yellow triangular sun using `GL_TRIANGLES`.
4. **House:** Create a house using:
   - A rectangular body using `GL_QUADS`.
   - A triangular roof using `GL_TRIANGLES`.
   - A rectangular door.
5. **Tree:** Create a tree using:
   - Brown rectangular trunk.
   - Green triangular leaves.
6. **Clouds:** Create clouds using multiple white rectangles.

Summary

- **OpenGL** draws; **GLUT** gives us the window; **PyOpenGL** lets us use both from Python.
- A shape is just a list of **corners** given in order.
- Colour must be chosen **before** drawing.
- The picture is repainted **again and again**, very fast.
- We paint on a hidden page and then **swap** it into view, so the image looks smooth.

That is all a “graphics program” really is.

Thank you

Questions are welcome